浙江大學

计算机应用数学(上) 课程论文
(2021-2022 春)

姓名 李国昌

学号 12121091

专业 计算机科学与技术

目录

1 引言 **3**

2 方法概述 **3**

2.1 连续词袋模型 3

2.1.1 上下文为一个词的词袋模型 3

2.1.2 多词语的上下文 6

2.2 Skip-Gram 模型 7

2.3 计算效率的优化 9

2.3.1 分层 softmax 9

2.3.2 负采样 11

3 实验结果 **12**

3.1 数据集 12

3.2 实验环境 12

3.3 实验内容 12

3.3.1 语料清洗 12

3.3.2 模型结构 13

3.4 实验效果 14

4 小结与讨论 **14**

1 引言

近几年，随着attention机制 [1]的提出，Transformer开始大规模应用于自然语言领域，随之而产生的大规模词嵌入模型BERT [2]以及ALBERT逐渐成为自然语言处理任务中进行词嵌入的主要方式。然而，在BERT大规模应用前， Word2vec [3]是自然语言处理任务中经常使用的词嵌入模型，相比于BERT的多层神经网络， Word2vec 的网络更浅，其解释性也更强，此外 Word2vec 所提出的词袋模型仍然对现在很多深度学习任务是有指导意义的。

近几年，对机器学习模型的可解释性逐渐成为了一个研究热点，本文所选的论文并非Word2vec的开山之作，而是一篇在arxiv但对 Word2vec 模型参数详细解释的文章。虽然文献 [3]没有发表，但引用量也达到了1000以上，其对 Word2vec 甚至神经网络的解释是极致详尽的，其提供了 Word2vec 模型的参数更新方程的详细推导和解释，包括原始的连续词袋 (CBOW) 和跳跃语法 (SG) 模型，以及高级优化技术，包括分层 softmax 和负采样。此外，其也提供了神经元网络和反向传播的基础知识。

2 方法概述

2.1 连续词袋模型

2.1.1 上下文为一个词的词袋模型

最简单的词袋模型(CBOW) [4]模型是 Mikolov 等人提出的，首先我们考虑最简单的情况，即上下文为一个词的情况。在这里我们假设每个上下文仅考虑一个词，也就是说我们给定一个词，模型就会预测一个词，其网络结构如图(1)所示。

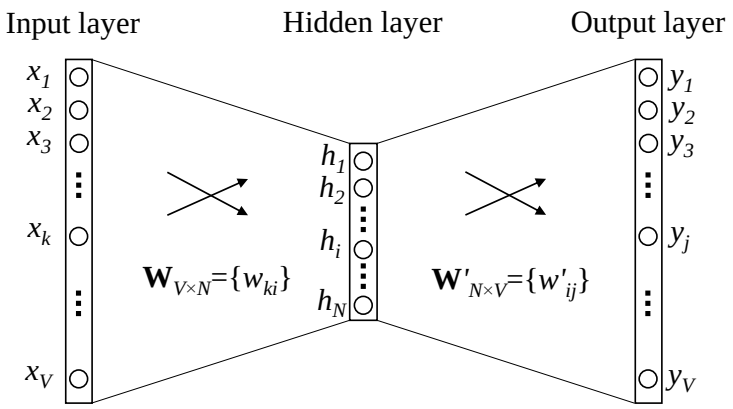

图 1: 上下文为一个词的连续词袋模型

如果将模型使用数学的形式表达出来，那么首先我们认为词表的大小为V，隐藏层的大小为N，且神经元之间是全连接的，输入向量为一个 one-hot 向量，即一个上下文中的词。One-hot 是将词表示为向量的一种方式，其首先需要提取文章中所有的词构建词表，这里我们假设词表的大小为V，因此每个One-hot向量的维度应该为V维，因此我们可以将 One-hot 向量表示为$\{x_1, \cdots, x_V\}$，其中仅有一个元素值为1，其他元素的值为0。我们将输入层与隐藏层之间的权重表示为$V \times N$的矩阵$\mathbf{W}$，隐藏层与输出层的权重表示为$N \times V$的矩阵$\mathbf{W}'$。对于隐藏层向量，可以通过(1)计算得到:

$$\mathbf{h} = \mathbf{W}^T\mathbf{x}, \tag{1}$$

再根据(2(其中Col_i))表示$\mathbf{W}^T$的第i列，即$\mathbf{W}$的第i行，由此我们可以将$\mathbf{W}$的第i行向量理解为其为输入层中第 i 个词的 N 维向量表达，我们将$\mathbf{W}$的行向量表示为$\mathbf{v}_w$。

$$\mathbf{W}^T \times \mathbf{x} = \sum_{i=1}^{V} x_i \times Col_i, \tag{2}$$

由此我们将(1)进一步推导可以得到(3)，其中 $\mathbf{v}_{w_I}$表示输入词w_I的 N 维向量表达，同时在这里我们也看到没有设置额外的激活函数:

$$\mathbf{h} = \mathbf{W}^T\mathbf{x} = \mathbf{W}^T_{(k,\cdot)} := \mathbf{v}^T_{w_I}, \tag{3}$$

从隐藏层前进到输出层的过程可以用(4)得到，实际上这个过程我们计算了词表中的每个词的分数u_j，因为u_j可以用(5)表示，其中，$\mathbf{v}'_{w_j}$为矩阵$\mathbf{W}'$的第j列，而矩阵$\mathbf{W}'$共有V列，正好对应了整个词表，也就是说，$\mathbf{W}'$对每个词的N维表示进行了打分，以便后续进行预测。

$$\mathbf{y} = \mathbf{W}'^T \times \mathbf{h}, \tag{4}$$

$$u_j = \mathbf{v}'^{\ T}_{w_j}\mathbf{h}, \tag{5}$$

随后我们使用$softmax$对计算结果进行预测并输出，即概率最大者为预测的下一个词的标签，这里的$softmax$可以解释为当输入词w_I的时候，输出词w_j的概率，即(6)所示，其中y_i为输出层第j个神经元的值:

$$p(w_j|w_I) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^{V}\exp(u_{j'})}, \tag{6}$$

如果我们将(3)与(5)带入(6)中可以得到(7)，通过这个公式可以比较直观的帮助我们理解这个词袋模型的含义。

$$p(w_j|w_I) = \frac{\exp\left(\mathbf{v}'^{\ T}_{w_j}\mathbf{v}_{w_I}\right)}{\sum_{j'=1}^{V}\exp\left(\mathbf{v}'^{\ T}_{w_{j'}}\mathbf{v}_{w_I}\right)} \tag{7}$$

为了下文方便解释，我们将$\mathbf{v}_{w_I}$叫做出入向量，而$\mathbf{v}'_{w_j}$叫做输出向量。根据我们以上所述，我们已经得到了输入到输出的计算过程，下一步我们将继续探究如何更新两个矩阵$\mathbf{W}$以及$\mathbf{W}'$的参数。

从输出层到隐藏层的参数更新 由于参数的更新使用了后向传播的方法，所以我们首先来推导输出层到隐藏层的参数更新过程，优化的目标应该是最大化(6)中的概率，其中j应该为j^*，即预测正确的词语w_j，由此我们可以得到以下(10)：

$$\begin{aligned}
\max p(w_O|w_I) &= \max y_{j^*} && (8)\\
&= \max \log y_{j^*} && (9)\\
&= u_{j^*} - \log\sum_{j'=1}^{V}\exp(u_{j'}) := -E, && (10)
\end{aligned}$$

$E = -\log p(w_O|w_I)$即为损失函数，其实这里的损失函数是交叉熵函数的一种特别形式，交叉熵损失的函数形式为$loss = -\sum_{i=1}^{n} t_i log y_i$，其中$t_i$为标签即真值，$y_i$为模型计算的预测值，对于本文的项目来说，由于使用了Ont-hot编码，当$i = j$时，即预测值等于$ground \quad truth$的时候，t_i为1，除此之外t_i为0，所以可以推导出损失函数E如(10)所示，根据损失函数与u_j之间的关系，我们对u_j求偏导可以得到：

$$\frac{\partial E}{\partial u_j} = y_j - t_j := e_j \tag{11}$$

如之前所述，当$j=j^*$时，$t_j=1$，此外$t_j=0$，为了方便表示，我们将y_j-t_j表示为e_j，根据参数更新的链式法则我们继续向后推导参数更新的过程，我们推导损失函数对矩阵$\mathbf{W}'$参数$w_{ij}{}'$的偏导有如下所示的过程：

$$\frac{\partial E}{\partial w'_{ij}}=\frac{\partial E}{\partial u_j}\cdot\frac{\partial u_j}{\partial w'_{ij}}=e_j\cdot h_i \tag{12}$$

由此，我们得到矩阵$\mathbf{W}'$参数更新的数学表达为：

$$w'_{ij}{}^{(\text{new})}=w'_{ij}{}^{(\text{old})}-\eta\cdot e_j\cdot h_i. \tag{13}$$

我们将其写为向量的形式，即如下所示：

$$\mathbf{v}'_{w_j}{}^{(\text{new})}=\mathbf{v}'_{w_j}{}^{(\text{old})}-\eta\cdot e_j\cdot\mathbf{h}\qquad\text{for } j=1,2,\cdots,V. \tag{14}$$

其中η为学习率，从参数更新方程中我们可以发现，如果要完全更新矩阵$\mathbf{W}'$的参数，我们需要遍历整个词表，另一方面，我们可以通过(14)可以理解参数更新的过程，当$y_i\ \geq\ t_i$时(由于y_i表示概率其值在0到1之间，所以这个情况下t_i的值为0)，这表示模型过度估计，这个时候我们需要减少对应的参数，以使得预测值更加靠近$ground\quad truth$，当然，具体减少的幅度还要根据学习率η调节；如果$y_i\ \leq\ t_i$时(由于y_i表示概率其值在0到1之间，所以这个情况下t_i的值为1)，这个时候我们预测值太小，所以我们需要调大对应的参数来使得其模型的计算结果更加靠近$ground\quad truth$。

从隐藏层到输入层的参数更新 接下来，我们来推导从隐藏层到输入层的参数更新过程，由于我们之前已经推导损失函数对u_j的偏导，接下来我们结合(5)推导损失函数对隐藏层参数h_i的偏导，推导如下：

$$\frac{\partial E}{\partial h_i}=\sum_{j=1}^{V}\frac{\partial E}{\partial u_j}\cdot\frac{\partial u_j}{\partial h_i}=\sum_{j=1}^{V}e_j\cdot w'_{ij}:=\text{EH}_i \tag{15}$$

$\text{EH}=[EH_1,EH_2,\cdots,EH_N]$是维度为$N$的向量，结合(15)中$\text{EH}_i$的计算方式，我们可以将EH向量改写成(16)中的形式，其中$\mathbf{v}_{w_i}{}'$表示矩阵$\mathbf{W}'$中的第i列，所以从EH的这种形式上我们可以EH的具体含义，即预测错误加权后的所有词的输出向量之和。

$$\text{EH}=e_1\mathbf{v}'_{w_1}+e_2\mathbf{v}'_{w_2}+\cdots+e_V\mathbf{v}'_{w_V} \tag{16}$$

由此，我们接下来就可以推导损失函数对矩阵$\mathbf{W}$中参数更新过程，推导如下有：

$$\frac{\partial E}{\partial w_{ki}}=\frac{\partial E}{\partial h_i}\cdot\frac{\partial h_i}{\partial w_{ki}}=\text{EH}_i\cdot x_k \tag{17}$$

进一步的我们可以将上述推导表示如下的形式为：

$$\frac{\partial E}{\partial\mathbf{W}}=\mathbf{x}\otimes\text{EH}=\mathbf{x}\text{EH}^T \tag{18}$$

由于输入向量$\mathbf{x}$为 One-hot向量，其只有一个元素值为1，因此$\frac{\partial E}{\partial\mathbf{W}}$中只有一行不为0，这一行为 EH^T，其他行为0，因此这里只写出输入词w_I所在行的参数更新过程，那么矩阵$\mathbf{W}$ 参数更新的过程可以表示为19：

$$\mathbf{v}_{w_I}^{(\text{new})}=\mathbf{v}_{w_I}^{(\text{old})}-\eta\text{EH}^T \tag{19}$$

由于w_I为唯一的输入词，所以矩阵$\mathbf{W}$中实际用到的只有$\mathbf{v}_{w_I}$这一个输出向量，所以我们可以直接将矩阵的参数更新写成是向量$\mathbf{v}_{w_I}$的更新。至此，本模型用到的两个参数的矩阵的更新推导以及最后的更新计

算方法已经如(14)以及(19)所示。在本文的引言部分，本文曾提到，本文的所描述的模型具有可解释性，那么接下来，我们结合(14)以及(19)来解释参数如此更新的实际含义。

随着我们迭代地输入一对上下文词语，参数也在迭代地更新，而且更新的效果是不断累计的。如果我们将这个问题映射成一个物理问题，对于一个词w，其输出向量会受到上下文中被输入且会输出w的词的输入向量的牵引，好像w的输出向量受到了这些输入向量的牵引力，受到牵引的w会“前进一段距离”，即下次再输入这些词时模型输出w的概率更大，这就是说模型的预测效果更加准确。同理，对于输入向量，其也受到其上下文词语的输出向量的牵引，这也使得这个输入词更容易预测出其下文的词语。最后，通过不断的迭代，所以输入输出向量能够达到一个比较平衡的状态，如果从物理学上来说，就是各个向量受到的牵引力的合力为0.

2.1.2 多词语的上下文

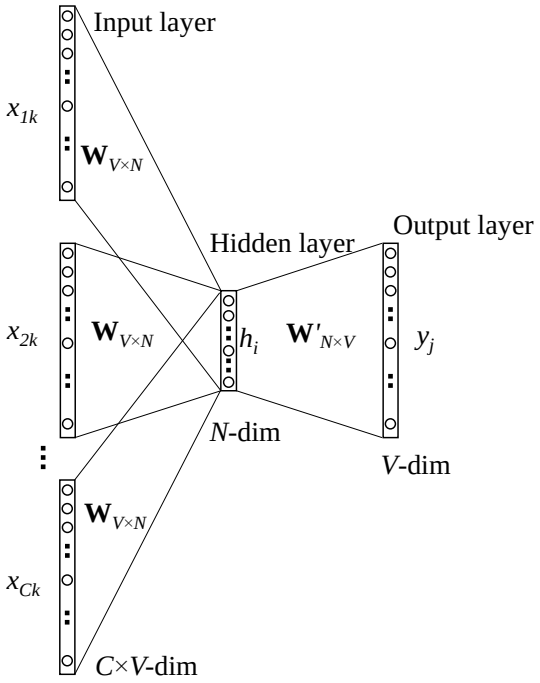

图 2: Continuous bag-of-word model

图2展示了一个上下文词语的 CBOW 模型，与上一小节上下文只有一个词的情况相比，多词语的模型隐藏层的输入多个词语的输入向量和的平均，其中C是输入词语的数量，$w_1, \cdots, w_C$表示上下文输入的词语：

$$\mathbf{h} = \frac{1}{C}\mathbf{W}^T(\mathbf{x}_1 + \mathbf{x}_2 + \cdots + \mathbf{x}_C) \tag{20}$$

$$= \frac{1}{C}(\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \cdots + \mathbf{v}_{w_C})^T \tag{21}$$

进而，我们结合上一小结的推导，损失函数E应该为：

$$
\begin{aligned}
E &= = -\log p(w_O|w_{I,1},\cdots,w_{I,C}) && (22)\\
&= -u_{j^*} + \log\sum_{j'=1}^{V}\exp(u_{j'}) && (23)\\
&= -\mathbf{v}'_{w_O}{}^T\cdot\mathbf{h} + \log\sum_{j'=1}^{V}\exp(\mathbf{v}'_{w_j}{}^T\cdot\mathbf{h}) && (24)
\end{aligned}
$$

同时，输出向量的参数更新方程与上下文一个词的情况也是一致的，即为：

$$
\mathbf{v}'_{w_j}{}^{(\text{new})} = \mathbf{v}'_{w_j}{}^{(\text{old})} - \eta\cdot e_j\cdot\mathbf{h} \qquad \text{for } j=1,2,\cdots,V. \tag{25}
$$

由于输入词不止一个，因此我们需要更新所有输入词的输入向量参数，为：

$$
\mathbf{v}_{w_{I,c}}^{(\text{new})} = \mathbf{v}_{w_{I,c}}^{(\text{old})} - \frac{1}{C}\cdot\eta\cdot\text{EH}^T \qquad \text{for } c=1,2,\cdots,C. \tag{26}
$$

2.2 Skip-Gram 模型

Skip-Gram模型 [5]也是Mikolov等人提出的，其结构如图3所示，其与上一节中的 CBOW 模型正好相反，目标词成为了输入，而上下文词语成为了输出，即通过一个词预测上下文多个词语，接下来我们将推导输入层与隐藏层之间的参数矩阵$\mathbf{W}$以及隐藏层与输出层之间的参数矩阵$\mathbf{W}'$的参数更新方程。

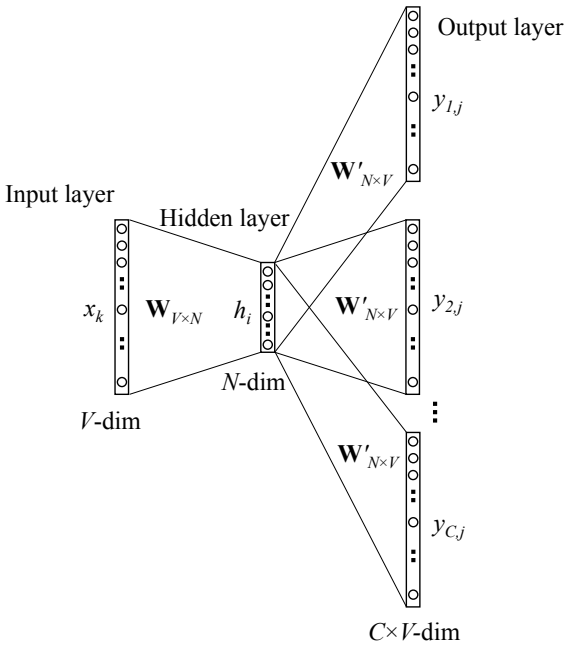

图 3: skip-gram 模型

首先，我们先正向地推导输入层到输出层的计算过程，由于输入层只有一个词语我们依然假设这个词语为w_I，因此隐藏层的向量 $\mathbf{h}$ 的计算过程为：

$$
\mathbf{h} = \mathbf{W}_{(k,\cdot)}^T := \mathbf{v}_{w_I}^T, \tag{27}
$$

由于输入向量仍为 One-hot 向量，因此计算得到的结果仍然的参数矩阵 $\mathbf{W}$ 中的一列，我们将其记作 $\mathbf{v}_{w_I}^T$ ，我们继续延续 CBOW 模型中对其的叫法，即输入向量。从隐藏层到输出层，我们仍先计算分数$u_{c,j}$，这个变量表示在第c个输出向量上的第j个词的分数。由于我们最后会输出C个向量，所以这里的

分数有两个下标来表示是第几个向量中的第几个词语的分数，由于隐藏层到所有输出向量的计算过程公用一个参数矩阵$\mathbf{W}'$，因此我们仍然使用$\mathbf{v}'_{w_j}$表示第j个词的输出向量，所以其计算过程如下：

$$u_{c,j} = u_j = \mathbf{v}'_{w_j}{}^T \cdot \mathbf{h}, \text{ for } c = 1, 2, \cdots, C \tag{28}$$

接下来就是$softmax$层，我们假设$w_{O,c}$就是第c个向量实际应该输出的词语，因此我们应该最大化输出这个的词的概率，即最大化以下概率：

$$p(w_{c,j} = w_{O,c}|w_I) = y_{c,j} = \frac{\exp(u_{c,j})}{\sum_{j'=1}^{V} \exp(u_{j'})} \tag{29}$$

进而，损失函数应该是最大化输出向量为实际的C个向量的概率，即我们需要将C个向量相乘并取最大化，其中， j_c^*表示第c个向量实际应该输出词的此表中的下标，因此有：

$$\begin{aligned} E &= -\log p(w_{O,1}, w_{O,2}, \cdots, w_{O,C}|w_I) && (30) \\ &= -\log \prod_{c=1}^{C} \frac{\exp(u_{c,j_c^*})}{\sum_{j'=1}^{V} \exp(u_{j'})} && (31) \\ &= -\sum_{c=1}^{C} u_{j_c^*} + C \cdot \log \sum_{j'=1}^{V} \exp(u_{j'}) && (32) \end{aligned}$$

接下来我们自右到左推导模型参数矩阵的更行方程，首先计算损失函数对分数$u_{c,j}$的导数，实际上由于输出的向量之间的u并无关联，所以这个计算结果与 CBOW 模型基本一致，只是需要额外标注是哪个输出向量的即可：

$$\frac{\partial E}{\partial u_{c,j}} = y_{c,j} - t_{c,j} := e_{c,j} \tag{33}$$

$e_{c,j}$ 我们仍然成为是预测错误的值(即为实际值与预测值之间的差)，为了标记方便我们将V维向量EI记作，$\mathrm{EI} = \{\mathrm{EI}_1, \cdots, \mathrm{EI}_V\}$，其中：

$$\mathrm{EI}_j = \sum_{c=1}^{C} e_{c,j} \tag{34}$$

接下来，我们就可以来推导参数矩阵$\mathbf{W}'$的参数更新方程：

$$\frac{\partial E}{\partial w'_{ij}} = \sum_{c=1}^{C} \frac{\partial E}{\partial u_{c,j}} \cdot \frac{\partial u_{c,j}}{\partial w'_{ij}} = \mathrm{EI}_j \cdot h_i \tag{35}$$

于是，我们可以将这个参数更新方程记作：

$$w'_{ij}{}^{(\text{new})} = w'_{ij}{}^{(\text{old})} - \eta \cdot \mathrm{EI}_j \cdot h_i \tag{36}$$

或者

$$\mathbf{v}'_{w_j}{}^{(\text{new})} = \mathbf{v}'_{w_j}{}^{(\text{old})} - \eta \cdot \mathrm{EI}_j \cdot \mathbf{h} \qquad \text{for } j = 1, 2, \cdots, V. \tag{37}$$

参数矩阵$\mathbf{W}$参数更新方程与上一小节中的模型基本相似，最后的结果需要将e_j替换为EI_j，于是有：

$$\mathbf{v}_{w_I}^{(\text{new})} = \mathbf{v}_{w_I}^{(\text{old})} - \eta \cdot \mathrm{EH}^T \tag{38}$$

其中EH可以记作：

$$\mathrm{EH}_i = \sum_{j=1}^{V} \mathrm{EI}_j \cdot w'_{ij}. \tag{39}$$

对于参数更新的方程的解释与上一小节最后的解释基本一致，在次便不再赘述。

2.3 计算效率的优化

我们上述的推导是非常直接的，中间没有经过任何的优化，我们根据我们的推导来观察模型的计算效率。(25) 与 (36) 分别是输入向量与输出向量的计算方式，输入向量的计算开销并不大，但是对于每次迭代更新输出向量时，我们需要遍历一边词表，计算预测输出，每次迭代需要遍历一遍词表的开销过于巨大，如果语料库比较大，那么词表的尺寸也是非常巨大的，这无疑会增加非常大的计算开销，所以本文接下来提出两个提高计算效率的方法：分层$softmax$和采样。

2.3.1 分层 softmax

Hierarchical softmax [6,7](即分层$softmax$)是一种计算$softmax$的高效计算方法，该模型使用二叉树来表示词汇表中的所有单词。 V 个单词必须是树的叶节点，可以证明有V-1个内部节点。对于每个叶子节点，存在从根到该节点的唯一路径，而这条路径是用来估计叶子节点所代表的词的概率，图4是这种树的一个例子：

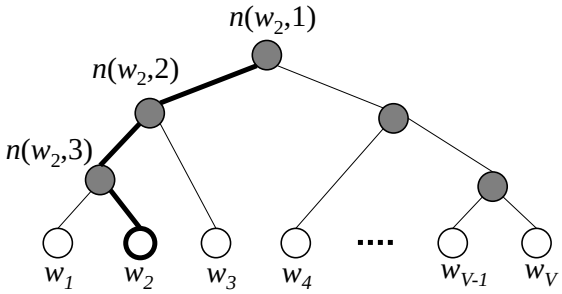

图 4: An example binary tree for the hierarchical softmax model. The white units are words in the vocabulary, and the dark units are inner units. An example path from root to w_2 is highlighted. In the example shown, the length of the path $L(w_2) = 4$. $n(w, j)$ means the j-th unit on the path from root to the word w.

在分层 $softmax$ 模型中，没有词的输出向量表示。相反，每个内部节点都有一个输出向量$\mathbf{v}'_{n(w,j)}$，一个词作为输出词的概率定义为：

$$p(w = w_O) = \prod_{j=1}^{L(w)-1} \sigma\left([\![n(w, j+1) = \mathrm{ch}(n(w, j))]\!] \cdot {\mathbf{v}'_{n(w,j)}}^T \mathbf{h}\right) \tag{40}$$

其中，$\mathrm{ch}(n)$是节点n的$left \quad child$，$\mathbf{v}'_{n(w,j)}$表示内部节点$n(w, j)$的向量表达(“输出向量”)，$\mathbf{h}$是隐藏层的输出值(在 skip-gram 模型中，$\mathbf{h} = \mathbf{v}_{w_I}$; 在 CBOW 模型， $\mathbf{h} = \frac{1}{C}\sum_{c=1}^{C} \mathbf{v}_{w_c}$)， $[\![x]\!]$ 是(41)定义的函数。

$$[\![x]\!] = \begin{cases} 1 & \text{if } x \text{ is true;} \\ -1 & \text{otherwise.} \end{cases} \tag{41}$$

这里每个内部节点实际上包含了一个激活函数$sigmod$函数$\sigma(u)$，其具体形式以及某些性质如下所示：

$$\sigma(u) = \frac{1}{1 + e^{-u}} \tag{42}$$

$$\sigma(-u) = 1 - \sigma(u) \tag{43}$$

$$\frac{d\sigma(u)}{du} = \sigma(u)\sigma(-u) \tag{44}$$

接下来，我们通过一个实际的例子来理解(40)是如何使用的。对于图4，我们想要计算 w_2 作为输出词的概率。我们将此概率定义为随机访问的概率，该随机访问从根结点开始，到所讨论的叶节点结束。在每个内部节点（包括根节点），我们需要分配向左和向右的概率，我们定义在内部节点 n 向左的概率为：

$$p(n, \text{left}) = \sigma\left({\mathbf{v}'_n}^T \cdot \mathbf{h}\right) \tag{45}$$

那么，访问右节点的概率就是概率和1减去访问左孩子的概率，在结合(43)，在内部节点 n 向右的概率为：

$$p(n, \text{right}) = 1 - \sigma\left({\mathbf{v}'_n}^T \cdot \mathbf{h}\right) = \sigma\left(-{\mathbf{v}'_n}^T \cdot \mathbf{h}\right) \tag{46}$$

根据图4中标出的访问到w_2的路径，将访问路径上节点的概率相乘就可以得到输出词为w_2的概率为：

$$\begin{aligned} p(w_2 = w_O) &= p\left(n(w_2, 1), \text{left}\right) \cdot p\left(n(w_2, 2), \text{left}\right) \cdot p\left(n(w_2, 3), \text{right}\right) && (47) \\ &= \sigma\left({\mathbf{v}'_{n(w_2,1)}}^T \mathbf{h}\right) \cdot \sigma\left({\mathbf{v}'_{n(w_2,2)}}^T \mathbf{h}\right) \cdot \sigma\left(-{\mathbf{v}'_{n(w_2,3)}}^T \mathbf{h}\right) && (48) \end{aligned}$$

通过(40)也可以很显然的推导出输出所有词的概率之和为1(按照树结构相加左右孩子节点的概率即可)：

$$\sum_{i=1}^{V} p(w_i = w_O) = 1 \tag{49}$$

接下来，我们需要继续推导分层$softmax$的输出向量的参数更新过程，我们从上下文为一个词的 CBOW 开始推导，上下文多词的 CBOW 以及 Skip-Gram 模型都可以类似地被推导出来，同时为了简化标记，我们定义了以下几种缩写：

$$[\![\cdot]\!] := [\![n(w, j+1) = \text{ch}(n(w, j))]\!] \tag{50}$$

$$\mathbf{v}'_j := \mathbf{v}'_{n_{w,j}} \tag{51}$$

首先，我们需要写出损失函数，这里使用的仍然是交叉熵函数，当然由于输出为一个词，所以这里仍为交叉熵函数的特殊形式：

$$E = -\log p(w = w_O | w_I) = -\sum_{j=1}^{L(w)-1} \log \sigma([\![\cdot]\!] {\mathbf{v}'_j}^T \mathbf{h}) \tag{52}$$

仿照上下文只有一个词的 CBOW 模型对参数更新方程的推导过程，我们接下来应该对u_j进行求导，但是这里没有u_j，因此我们对 $\mathbf{v}'_j\mathbf{h}$ 这个整体进行求导有：

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{v}'_j \mathbf{h}} &= \left(\sigma([\![\cdot]\!] {\mathbf{v}'_j}^T \mathbf{h}) - 1\right) [\![\cdot]\!] && (53) \\ &= \begin{cases} \sigma({\mathbf{v}'_j}^T \mathbf{h}) - 1 & ([\![\cdot]\!] = 1) \\ \sigma({\mathbf{v}'_j}^T \mathbf{h}) & ([\![\cdot]\!] = -1) \end{cases} && (54) \\ &= \sigma({\mathbf{v}'_j}^T \mathbf{h}) - t_j && (55) \end{aligned}$$

如果 $[\![\cdot]\!] = 1$ 则$t_j = 1$ 否则 $t_j = 0$，接下来我们可以求损失函数E对参数向量$\mathbf{v}'_j\mathbf{h}$的导数：

$$\frac{\partial E}{\partial \mathbf{v}'_j} = \frac{\partial E}{\partial \mathbf{v}'_j \mathbf{h}} \cdot \frac{\partial \mathbf{v}'_j \mathbf{h}}{\partial \mathbf{v}'_j} = \left(\sigma({\mathbf{v}'_j}^T \mathbf{h}) - t_j\right) \cdot \mathbf{h} \tag{56}$$

最后可以得出输出向量的参数更新过程为：

$$\mathbf{v}'^{(\text{new})}_j = \mathbf{v}'^{(\text{old})}_j - \eta \left(\sigma({\mathbf{v}'_j}^T \mathbf{h}) - t_j\right) \cdot \mathbf{h} \tag{57}$$

我们可以将 $\sigma({\mathbf{v}'_j}^T \mathbf{h}) - t_j$ 理解为内部节点 $n(w,j)$ 的预测误差。每个内部节点的“任务”是在随机访问中预测它应该访问左孩子还是右孩子。$t_j = 1$ 表示实际应该是跟随左孩子，$t_j = 0$ 意味着它应该跟随右孩子，$\sigma({\mathbf{v}'_j}^T \mathbf{h})$ 是预测结果。对于一个训练实例，如果内部节点的预测非常接近$ground \quad truth$，那么它的向量表示$\mathbf{v}'_j$ 将移动很小；否则 $\mathbf{v}'_j$ 将向适当的方向(离 $\mathbf{h}$ 更近或更远)移动，以减少这种情况下的预测误差。此更新方程可用于 CBOW 和 skip-gram 模型。当用于 skip-gram 模型时，我们需要对输出上下文中的每个C词重复此更新过程。

接下来需要推导输入向量的参数更新过程，我们仍然需要先求损失函数E对$\mathbf{h}$的偏导，有：

$$\begin{aligned}
\frac{\partial E}{\partial \mathbf{h}} &= \sum_{j=1}^{L(w)-1} \frac{\partial E}{\partial \mathbf{v}'_j \mathbf{h}} \cdot \frac{\partial \mathbf{v}'_j \mathbf{h}}{\partial \mathbf{h}} && (58)\\
&= \sum_{j=1}^{L(w)-1} \left(\sigma({\mathbf{v}'_j}^T \mathbf{h}) - t_j\right) \cdot \mathbf{v}'_j && (59)\\
&:= \text{EH} && (60)
\end{aligned}$$

将得到的 EH 带入(26)即可得到输入向量的参数更新过程，对于 skip-gram 模型，我们需要为 skip-gram 上下文中的每个词计算一个 EH 值，并将 EH 值之和代入式(38)，得到输入向量的更新过程。

从更新方程中，我们可以看到每个上下文词的每个训练实例的计算复杂度从 $O(V)$ 降低到 $O(\log(V))$，这在计算效率上是一个很大的提升，同时我们保证了参数的数量与之前大致相同（内部节点的 $V-1$ 个向量与单词的原始V个输出向量相比）。

2.3.2 负采样

负采样的想法比分层 $softmax$ 更直接：为了解决每次迭代需要更新太多输出向量的困难，我们只更新它们的一个样本。显然，输出词（即$ground \quad truth$或正样本）应该保留在我们的样本中并得到更新，我们需要将一些词作为负样本进行采样（因此此方法称为负采样）。采样过程需要遵循一定的概率分布，我们将此分布称为噪声分布，并将其表示为 $P_n(w)$，我们可以凭经验确定一个好的分布 [4]。

在 word2vec 中，作者认为以下简化的训练目标能够产生高质量的词嵌入，而不是使用产生明确定义的后多项式分布的负采样形式 [5]：

$$E = -\log \sigma({\mathbf{v}'_{w_O}}^T \mathbf{h}) - \sum_{w_j \in \mathcal{W}_{\text{neg}}} \log \sigma(-{\mathbf{v}'_{w_j}}^T \mathbf{h}) \tag{61}$$

w_O是输出词(即正样本)，$\mathbf{v}'_{w_O}$是输出向量，$\mathbf{h}$是隐藏层的输出值，在 CBOW 模型中，$\mathbf{h} = \frac{1}{C}\sum_{c=1}^{C} \mathbf{v}_{w_c}$；在 skip-gram 模型中，$\mathbf{h} = \mathbf{v}_{w_I}$。此外，$\mathcal{W}_{\text{neg}} = \{w_j | j = 1, \cdots, K\}$基于 $P_n(w)$ 采样的单词集合，即负样本。

为了得到负采样下词向量的更新方程，我们仍然按照之前的方法进行求偏导，首先是对u_j：

$$\begin{aligned}
\frac{\partial E}{\partial {\mathbf{v}'_{w_j}}^T \mathbf{h}} &= \begin{cases} \sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) - 1 & \text{if } w_j = w_O \\ \sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) & \text{if } w_j \in \mathcal{W}_{\text{neg}} \end{cases} && (62)\\
&= \sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) - t_j && (63)
\end{aligned}$$

其中 t_j 是单词 w_j 的标签，当 w_j 为正样本时，$t=1$，否则$t=0$。接下来我们对单词 w_j 的输出向量求 E 的导数：

$$\frac{\partial E}{\partial \mathbf{v}'_{w_j}} = \frac{\partial E}{\partial {\mathbf{v}'_{w_j}}^T \mathbf{h}} \cdot \frac{\partial {\mathbf{v}'_{w_j}}^T \mathbf{h}}{\partial \mathbf{v}'_{w_j}} = \left(\sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) - t_j\right) \mathbf{h} \tag{64}$$

于是可以得到输出向量的参数更新方程：

$${\mathbf{v}'_{w_j}}^{(\text{new})} = {\mathbf{v}'_{w_j}}^{(\text{old})} - \eta \left(\sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) - t_j\right) \mathbf{h} \tag{65}$$

由于这个更新过程只需要对$w_j \in \{w_O\} \cup \mathcal{W}_{\text{neg}}$中的词进行更新即可，而不是整个词表中的所有词，以此来提高模型的计算效率。接下来我们继续对隐藏层的输出$\mathbf{h}$就偏导有：

$$\frac{\partial E}{\partial \mathbf{h}} = \sum_{w_j \in \{w_O\} \cup \mathcal{W}_{\text{neg}}} \frac{\partial E}{\partial {\mathbf{v}'_{w_j}}^T \mathbf{h}} \cdot \frac{\partial {\mathbf{v}'_{w_j}}^T \mathbf{h}}{\partial \mathbf{h}} \tag{66}$$

$$= \sum_{w_j \in \{w_O\} \cup \mathcal{W}_{\text{neg}}} \left(\sigma({\mathbf{v}'_{w_j}}^T \mathbf{h}) - t_j\right) \mathbf{v}'_{w_j} := \text{EH} \tag{67}$$

通过将 EH 代入式 (26)，我们得到了 CBOW 模型输入向量的更新方程。对于skip-gram模型，我们需要为skip-gram上下文中的每个词计算一个EH值，并将EH值之和代入式(38)，得到输入向量的更新方程。

3 实验结果

3.1 数据集

Word2vec 模型最早由Google提出，其公开了 Word2vec 的源码，在源码中其使用了$text8$作为训练词向量模型的数据集。因此本文也使用$text8$这个数据集作为训练的数据集，这个数据集里包含了大量从维基百科收集到的英文语料。$text8$来源于$enwiki8$，而$enwiki8$最早是用来做文本压缩的。简单说来，$enwiki8$是从$wikipedia$上爬取的前100000000个字符；而$text8$就是把这些字符当中各种奇怪的符号啊，非英文字符全都去掉，再把大写字符转化成小写字符，把数字转化成对应的英语单词之后，得到的数据集。

3.2 实验环境

- **操作系统**: Windows 10
- **编程语言**: Python
- **深度学习框架**: Pytorch
- **GPU**: Nvidia GTX 1660(6G)

3.3 实验内容

3.3.1 语料清洗

首先需要对数据进行简单的清洗，由于$text8$数据集单词之间是用空格分割，所以我们不要进行分句的处理，我们首先使用$nltk$对语料进行分词的操作，分词之后我们从$nltk$库中下载英语停用词，随后将语料库中的停用词去除，最后我们得到的语料库的大小为10888361。

过于庞大的语料库，需要更大的模型进行训练，这对GPU的显存有非常大的要求，因此我们这里根据我们的GPU选择语料库中的部分语料进行训练，这里本实验选择了 2000 个词语进行训练。

3.3.2 模型结构

以下的图5与6所示的网络结构分别为 CBOW 与 Skip-Gram 的网络结构，本文将上下文输入词语的数量设置为10，嵌入层的大小设置为50，隐藏层的大小设置为512，随后每个模型分别训练80个epoch。

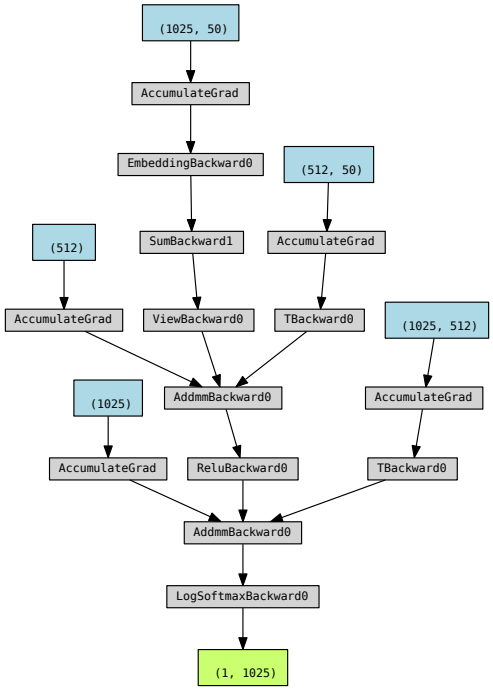

图 5: CBOW模型结构

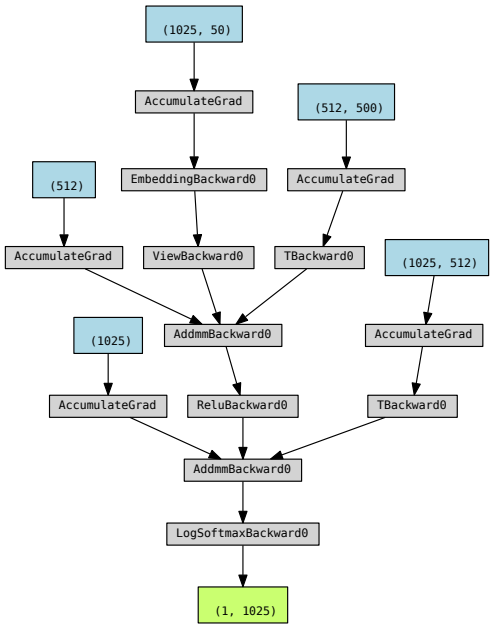

图 6: Skip-Gram模型结构

3.4 实验效果

图7与8分别为训练过程中损失函数值的变化过程，可以看到到训练的最后模型基本得到收敛。

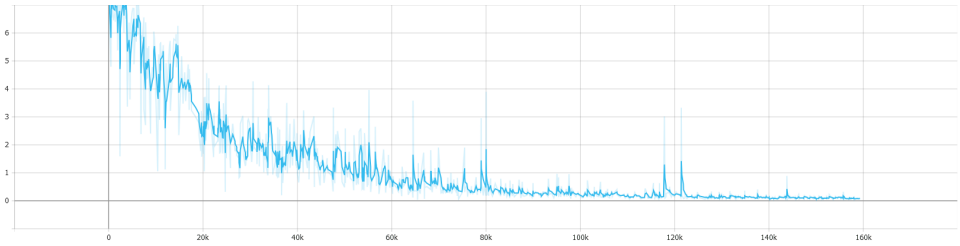

图 7: CBOW模型损失函数值

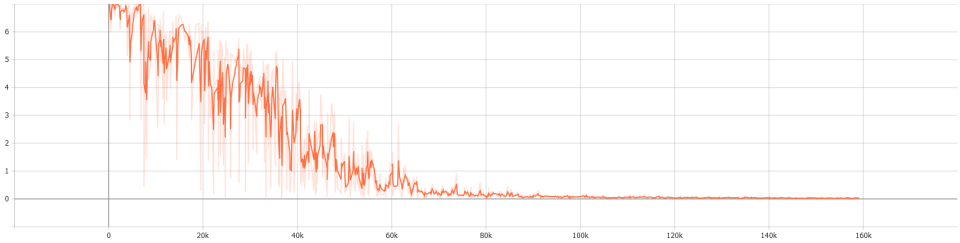

图 8: Skip-Gram模型损失函数值

我们使用训练好的词向量，输出词语“nearly”的20维词嵌入表达如下图所示，本文所用的代码和小数据集在本论文的同级目录下。

```
print(model.embeddings.weight[word_to_ix["nearly"]])
[38]  ✓ 0.6s
···  tensor([ 2.3162,  1.0948, -0.9804,  1.0768,  0.6072, -0.0932, -0.4718, -0.4824,
        -1.6976,  1.0703,  1.4242, -0.2210, -0.6059, -1.4530, -0.9254, -1.2532,
         1.9123, -2.0408,  1.8152,  0.1881], device='cuda:0',
       grad_fn=<SelectBackward0>)
```

图 9: 词“nearly”的词嵌入表达

4 小结与讨论

本文通过介绍了文献 [3]中提出的模型，介绍了 Word2vec 这一经典的词嵌入模型。在 BERT 等大型预训练语言模型的今天， Word2vec 的效果并不显得那么优秀，但是其模型的解释性是非常强的。本文分别介绍了 CBOW 与 Skip-Gram 两个 Word2vec 模型的运行过程，推导了每个模型的参数更新方程，并根据这些参数更新方程对这些更新方程的实际意义给出了一个非常合理的解释，也从另一个方面证明了模型训练及使用的可行性。最后，通过实现这两个模型来进一步加深对模型的理解，在模型网络结构越来越复杂的今天，追求的模型的可解释性显得非常重要，这对于更加合理的调整模型的结构具有非常好的指导意义而不是盲目的随机调整模型参数获得更好的效果，希望本文能够为机器学习的可解释性做出一点贡献。

参考文献

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA* (I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, eds.), pp. 5998–6008, 2017.

[2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)* (J. Burstein, C. Doran, and T. Solorio, eds.), pp. 4171–4186, Association for Computational Linguistics, 2019.

[3] X. Rong, "word2vec parameter learning explained," *CoRR*, vol. abs/1411.2738, 2014.

[4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings* (Y. Bengio and Y. LeCun, eds.), 2013.

[5] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States* (C. J. C. Burges, L. Bottou, Z. Ghahramani, and K. Q. Weinberger, eds.), pp. 3111–3119, 2013.

[6] A. Mnih and G. E. Hinton, "A scalable hierarchical distributed language model," in *Advances in Neural Information Processing Systems 21, Proceedings of the Twenty-Second Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 8-11, 2008* (D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, eds.), pp. 1081–1088, Curran Associates, Inc., 2008.

[7] F. Morin and Y. Bengio, "Hierarchical probabilistic neural network language model," in *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics, AISTATS 2005, Bridgetown, Barbados, January 6-8, 2005* (R. G. Cowell and Z. Ghahramani, eds.), Society for Artificial Intelligence and Statistics, 2005.